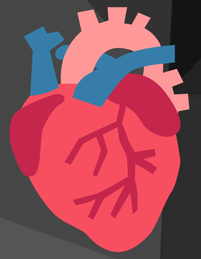
INFO 617 TEXT ANALYTICS

Text Classification of Medical Conversations

Presented by :
Shreya Mishra

PROJECT OVERVIEW

Objective: Develop a classification model to categorize medical conversations into 15 specific intent categories based on Social Support Theory.



Emotional Support (e.g., GREET, THANK, WAIT, WISH, RECEIVE, FUTURE_SUPPORT, CONSOLE, REMIND)

Informational Support (QUESTION, DIAGNOSE, TREAT, REFERRAL, REPEAT, EXPLAIN, REQUEST_INFORMATION)

Why Care?

Improve clarity, patient satisfaction, and healthcare outcomes

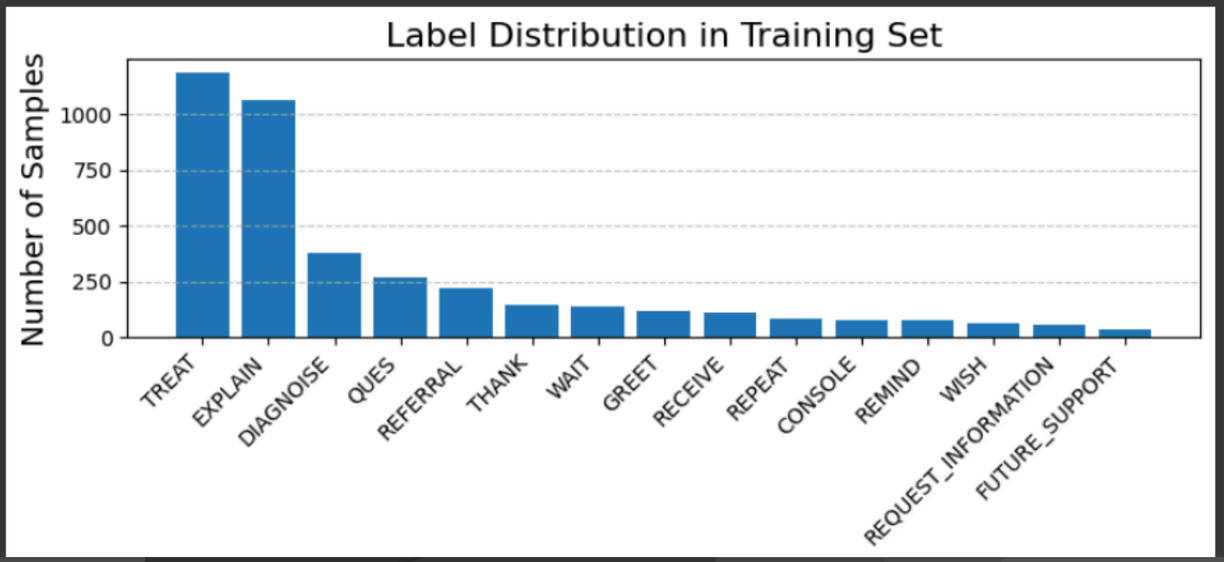
4,030 labeled sentences

Dataset Size

COMPREHENSIVE METHODOLOGY & WORKFLOW

1. Data Prep- (Preprocessing , Augmentation)
2. Class Imbalance Strategies (Oversampling Techniques,Loss Functions)
3. Features & Embeddings Static Embeddings, Contextual Embeddings
4. Models Built (ML, CNN, LSTM, Transformers)
5. Tuning & Validation (Hyperparameters, Early Stopping)
6. Evaluation & Selection (Test Accuracy, Macro F1-score)

CLASS LABELS DISTRIBUTION



IMBALANCED CLASS LABELS

TACKLED IT WITH WEIGHTED CROSS ENTROPY LOSS

```
# STEP 3: (NEW) Create weighted loss function
loss_fn = nn.CrossEntropyLoss(weight=class_weights)

# STEP 4: Initialize model and optimizer
cnn_model, optimizer = initilize_model(
    pretrained_embedding=embeddings,
    freeze_embedding=False,
    vocab_size=len(word2idx),
    embed_dim=embeddings.shape[1],
    num_classes=len(le.classes_),     # from LabelEncoder
    learning_rate=0.25,
    dropout=0.5
)

# STEP 5: Train the model
train(cnn_model, optimizer, train_loader, val_loader, epochs=20)
```

ALGORITHMS WE EXPLORED

Traditional ML Models :

1. Logistic Regression
2. Support Vector Machine (SVM)
3. Random Forest Classifier

Deep Learning Models :

1. Convolutional Neural Network (CNN)
   (Static and Non-static embeddings)
2. Long Short-Term Memory (LSTM)
   (Uni- and Bi-directional)
3. Transformer-based Models:
   - DistilBERT
   - FreezeBERT
   - DeBERTa
   - BioClinicalBERT

FEATURES EXTRACTED AND UTILIZED IN THE MODEL

- TF-IDF Vectors used for Logistic Regression, SVM, Random Forest.
- Word Embeddings (FastText) for CNN and LSTM models.
- BERT Tokenizer embeddings used for BERT.

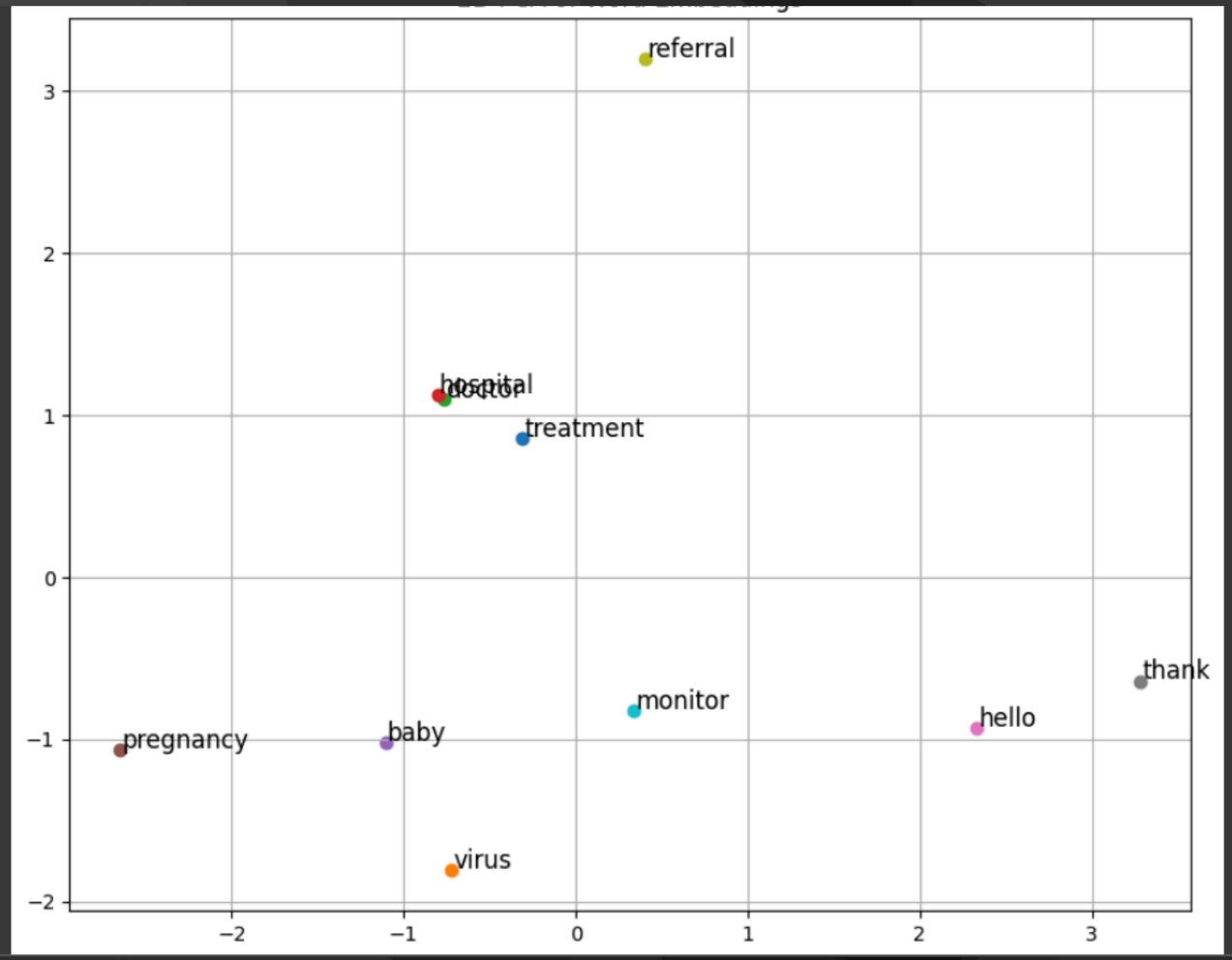
```
!mkdir -p fastText
!wget -O fastText/crawl-300d-2M.vec.zip https://dl.fbaipublicfiles.com/fasttext/vectors-english/crawl-300d-2M.vec.zip
!unzip fastText/crawl-300d-2M.vec.zip -d fastText
```

```
--2025-04-28 20:03:36--  https://dl.fbaipublicfiles.com/fasttext/vectors-english/crawl-300d-2M.vec.zip
Resolving dl.fbaipublicfiles.com (dl.fbaipublicfiles.com)... 108.138.94.49, 108.138.94.118, 108.138.94.128, ...
Connecting to dl.fbaipublicfiles.com (dl.fbaipublicfiles.com)|108.138.94.49|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1523785255 (1.4G) [application/zip]
Saving to: ‘fastText/crawl-300d-2M.vec.zip’

fastText/crawl-300d 100%[===================>]   1.42G   234MB/s    in 9.2s

2025-04-28 20:03:45 (158 MB/s) - ‘fastText/crawl-300d-2M.vec.zip’ saved [1523785255/1523785255]

Archive:  fastText/crawl-300d-2M.vec.zip
replace fastText/crawl-300d-2M.vec? [y]es, [n]o, [A]ll, [N]one, [r]ename:
```

EMBEDDINGS ANALYSIS

```
FT_PATH = "fastText/crawl-300d-2M.vec"
embeddings_np = load_pretrained_vectors(word2idx, FT_PATH)
embeddings = torch.tensor(embeddings_np, dtype=torch.float32)
```

```
✅ Loaded 5053 vectors from fastText/crawl-300d-2M.vec
```



HOW THESE FEATURES ARE USED IN THE MODELS

- TF-IDF vectors → direct input to traditional ML models.
- FastText embeddings → initialized CNN(1DCNN, Convolutional and pooling layers)
- LSTM layers(Bi-Directional LSTM) - The LSTM processes embeddings sequentially, remembering important context and using the final hidden state to predict the medical label
- BERT embeddings → BERT applies self-attention to embeddings, learning contextual relationships across the sentence,

MODEL RESULTS AND COMPARISON

Traditional ML:

Logistic Regression Model Accuracy Results:

```
# Predict on Test Set (Logistic Regression)
y_pred_test_logreg = log_reg.predict(X_test_tfidf)
logreg_test_acc = accuracy_score(test_labels, y_pred_test_logreg)

print(f"Logistic Regression Test Accuracy: {logreg_test_acc*100:.2f}%")
```

```
Logistic Regression Test Accuracy: 67.65%
```

Test Accuracy: 67.65%

SVM Model Accuracy Results:

```
# Predict on Test Set (SVM)
y_pred_test_svm = svm_model.predict(X_test_tfidf)
svm_test_acc = accuracy_score(test_labels, y_pred_test_svm)

print(f"SVM Test Accuracy: {svm_test_acc*100:.2f}%")
```

```
SVM Test Accuracy: 72.37%
```

Test Accuracy: 72.37%

Random Forest Model Accuracy Results:

```
Random Forest Test Accuracy: 64.87%
```

Predicted

	precision	recall	f1-score	support
CONSOLE	1.00	0.67	0.80	27
DIAGNOISE	0.74	0.22	0.34	119
EXPLAIN	0.55	0.72	0.62	316
FUTURE_SUPPORT	0.75	0.38	0.50	8
GREET	0.96	0.93	0.95	29
QUES	0.79	0.60	0.68	109
RECEIVE	0.94	0.86	0.90	37
REFERRAL	0.77	0.31	0.44	74
REMIND	1.00	0.62	0.76	21
REPEAT	0.00	0.00	0.00	30
REQUEST_INFORMATION	0.25	0.05	0.08	20
THANK	0.95	0.98	0.97	43
TREAT	0.56	0.83	0.67	281
WAIT	0.97	0.75	0.85	52
WISH	1.00	0.90	0.95	21
accuracy			0.65	1187
macro avg	0.75	0.59	0.63	1187
weighted avg	0.67	0.65	0.62	1187

Test Accuracy: 64.87%

MODEL RESULTS AND COMPARISON

Deep Learning Models:

LSTM Model Accuracy Results

Epoch	Train Loss	Val Loss	Val Acc	Elapsed
1	0.670233	1.184896	70.15	89.48
2	0.624505	1.132808	71.04	83.12
3	0.554846	1.244203	70.15	85.11
4	0.502999	1.328269	67.04	93.82
5	0.481722	1.287065	68.81	85.15
6	0.409068	1.365271	64.89	88.40
7	0.376296	1.399057	67.56	90.02
8	0.332961	1.400945	71.04	88.10
9	0.319219	1.433249	66.67	91.89
10	0.249532	1.672524	70.37	94.77

Training complete! Best accuracy: 71.04%.

Test Accuracy: 71.04%

CNN Model Accuracy Results:

Summary of Model Performances:

	Model	Test Accuracy (%)	Test Loss
0	Original CNN	73.578827	0.796784
2	CNN-static	73.020269	0.795674
3	CNN-nonstatic	72.686935	0.801812
1	CNN-rand	69.486485	1.020100

Test Accuracy: 73.57%

MODEL RESULTS AND COMPARISON

Transformer Model:

DeBERTa Large achieved highest performance:
Test Accuracy: 83.57%

Model	Test Accuracy	Validation Accuracy
Logistic Regression	67.65%	69.48%
Support Vector Machine (SVM)	72.37%	72.95%
Random Forest Classifier	64.87%	68.98%
Convolutional Neural Network(CNN)	73.57%	75.85%
Long Short-Term Memory(LSTM)	71.04%	71.04%
DeBERTa	83.57%	83.57%
DistilBERT	80.00%	79.95%
FreezeBERT	80.45%	79.16%
DeBERTa	82.65%	82.39%
BioClinicalBERT	76%	76.41%

Test Accuracy: 0.8265

Classification Report:

	precision	recall	f1-score	support
CONSOLE	0.61	0.93	0.74	27
DIAGNOISE	0.77	0.75	0.76	119
EXPLAIN	0.80	0.76	0.78	316
FUTURE_SUPPORT	1.00	0.75	0.86	8
GREET	1.00	0.97	0.98	29
QUES	0.99	0.97	0.98	109
RECEIVE	0.95	0.97	0.96	37
REFERRAL	0.68	0.96	0.80	74
REMIND	0.70	0.90	0.79	21
REPEAT	0.63	0.57	0.60	30
REQUEST_INFORMATION	0.71	0.60	0.65	20
THANK	0.91	0.95	0.93	43
TREAT	0.86	0.79	0.82	281
WAIT	0.93	0.96	0.94	52
WISH	0.95	0.95	0.95	21
accuracy			0.83	1187
macro avg	0.83	0.85	0.84	1187
weighted avg	0.83	0.83	0.83	1187

Model	Test Accuracy	Validation Accuracy
DistilBERT	80.00%	79.95%
FreezeBERT	80.45%	79.16%
DeBERTa	82.65%	82.39%
BioClinicalBERT	76%	76.41%
DeBERTa Large	83.57%	83.57%

BEST MODEL PERFORMANCE

Key Results:

- Test Accuracy: 83.57%
- Macro F1 Score: 85%

Validation Highlights:

- Stable validation accuracy (~83–84%) across epochs.
- Consistent generalization with early stopping

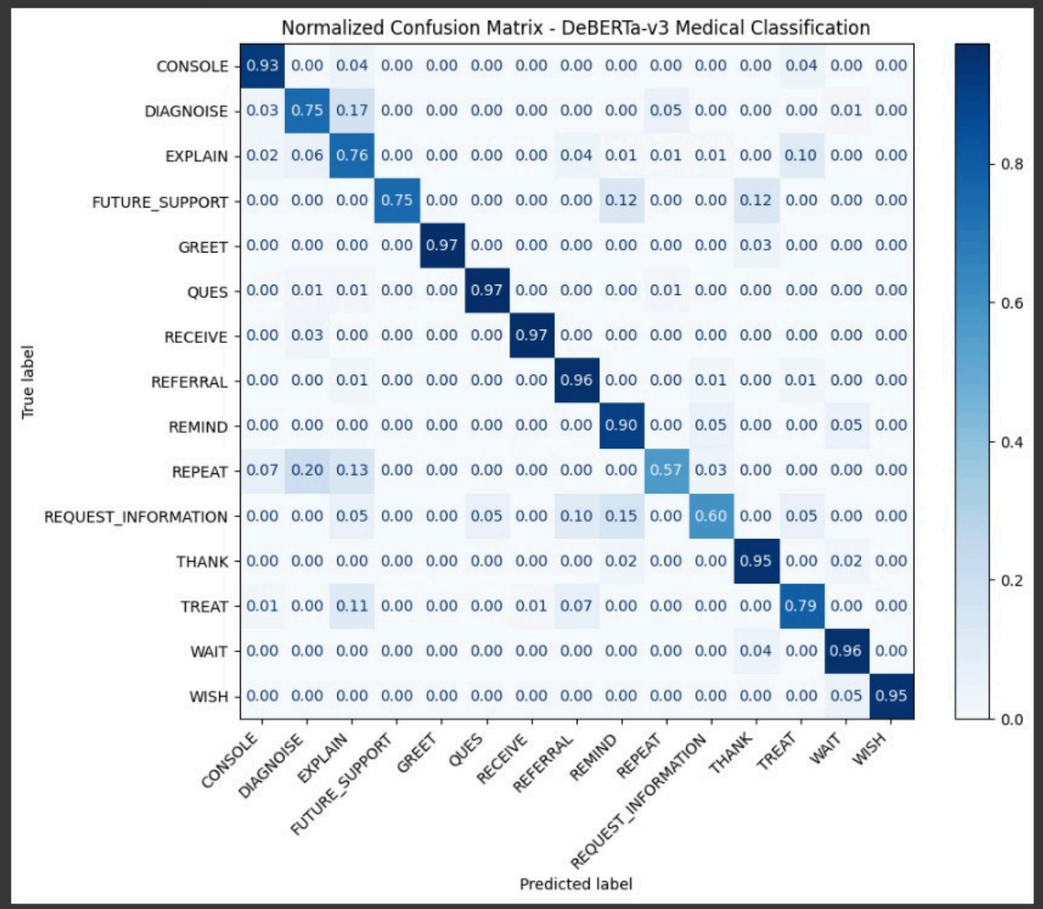
Performance Highlights:

- High F1-scores in key intents like GREET, THANK, and REQUEST_INFORMATION
- Focal Loss improved class balance
- DeBERTa captured deeper conversational meaning, outperforming ML and deep learning baselines

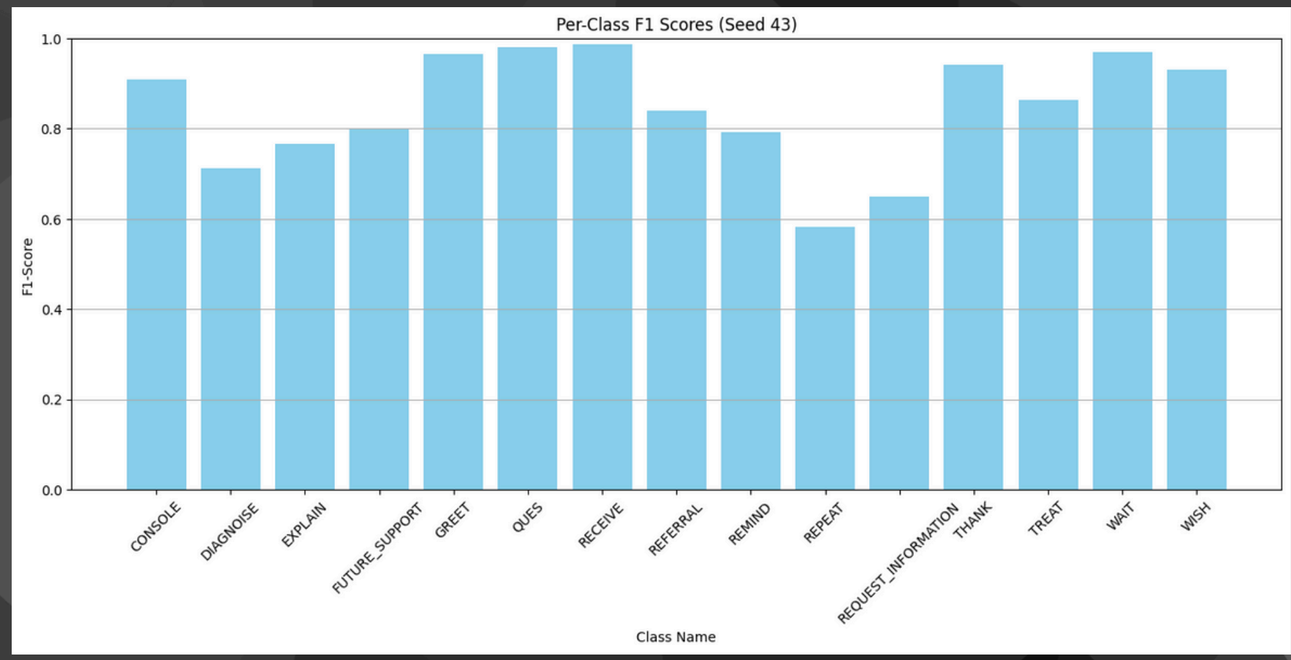
Key Takeaway:

- Fine-tuned **DeBERTa-v3-large** captured deep contextual meaning, significantly outperforming traditional models like **Logistic Regression, SVM,** and even deep learning **baselines (CNN, LSTM).**

Model Performance Analysis



Normalized Confusion Matrix (DeBERTa-v3)



Model Precision Across Categories

DEMO OF PREDICTING THE LABEL

```
# Typing loop
while True:
    user_input = input("  Type a medical sentence (or type 'exit' to stop): ")
    if user_input.lower() == 'exit':
        print(" Exiting prediction loop. Great work!")
        break

    prediction = predict_single_sentence(user_input, model, tokenizer)
    print(f" Predicted Label: {prediction}\n")
```

```
...    Type a medical sentence (or type 'exit' to stop):
```

SUMMARY AND FINAL INSIGHTS

Key Takeaways:

- Explored Traditional ML, Deep Learning, and Transformer-based models for text classification.
- Contextual embeddings (BERT family) captured sentence meaning far better than TF-IDF or static embeddings.
- Handling class imbalance through Weighted Loss and Focal Loss was crucial for achieving balanced performance.

Conclusion:

- Fine-tuned DeBERTa-v3-large achieved 83.57% Test Accuracy and 85% Macro F1 Score.
- Validation accuracy remained stable (~83–84%), showing strong generalization.
- Transformers outperformed traditional models by delivering deeper understanding of medical conversation intents.

THANK YOU